AI-Driven Adaptive Diagnostic Engine
1-Dimensional Adaptive Testing Prototype

Name: Kishan Bhalani
Tech Stack: Python, FastAPI, MongoDB, Groq API

Problem Statement

Problem:

- Most traditional quizzes give the same set of questions to every student. However, students have different skill levels, so a fixed quiz cannot accurately measure their true ability.
- If the questions are too easy, the test does not challenge the student. If they are too difficult, the student may struggle and lose confidence.

Goal:

- The goal of this project is to build an **adaptive testing system** that can adjust the difficulty of questions based on the student's performance.
- By doing this, the system can better estimate the student's ability level and identify the topics where the student needs more practice.

System Overview

- This system works by dynamically selecting questions based on how the student performs during the quiz.
- First, the student starts a quiz session.
  The system selects a question based on the student's current estimated ability level.
- After the student answers the question, the system checks whether the answer is correct or incorrect.
  Based on this result, the system updates the student's ability score.
- The next question is then selected with a difficulty level that matches the updated ability score.
- Once the quiz ends, the system analyzes the student's incorrect answers and identifies weak topics.
  These weak topics are then used to generate a personalized learning plan using an AI model.

Tech Stack

- For this project, I used **Python with FastAPI** to build the backend API. FastAPI was chosen because it is lightweight, easy to use, and automatically generates interactive API documentation.
- For the database, I used **MongoDB**, which is well suited for storing flexible data structures like quiz questions with metadata such as topic, difficulty, and tags.
- To run the API server, I used **Uvicorn**, which is a fast ASGI server designed for Python web applications.
- For the AI component, I integrated the **Groq API**, which uses a large language model to generate personalized learning recommendations based on the student's performance.

Database Design

- The system stores quiz questions in a MongoDB collection.
- Each question contains several important pieces of information such as the question text, available answer options, the correct answer, the difficulty level, and the topic.
- The difficulty value ranges from **0.1 to 1.0**, where lower values represent easier questions and higher values represent more difficult questions.
- This metadata allows the adaptive algorithm to select questions that are appropriate for the student's current ability level.
- By organizing the data in this way, the system can easily filter and retrieve questions based on difficulty and topic.

User Session Model

- To track a student's progress during the quiz, the system uses a **UserSession collection**.
- Each session stores information such as the session ID, the student’s current ability score, and the answers submitted during the quiz.
- The ability score represents the system’s estimate of the student’s skill level.
- At the beginning of the quiz, the ability score starts at a neutral value.
  As the student answers questions, the score is updated based on whether the answers are correct or incorrect.
- This allows the system to continuously adjust the difficulty of the questions being asked.

Adaptive Algorithm

- The adaptive algorithm is the core part of this system.
- At the start of the quiz, the student begins with a **baseline ability score of 0.5**.
- When a student answers a question correctly, it indicates that the question may have been too easy.
  In this case, the system slightly increases the ability score and selects a more difficult question next.
- If the student answers incorrectly, it indicates that the question may have been too difficult.
  The system then decreases the ability score and selects an easier question.
- By repeating this process, the system gradually moves toward a difficulty level that matches the student's true ability.

Adaptive Algorithm Flow

- The process begins when the student starts the quiz.
- The system assigns an initial ability score and selects a question that matches this difficulty level.
- The student answers the question, and the system evaluates whether the answer is correct.
- If the answer is correct, the ability score increases slightly.
  If the answer is incorrect, the ability score decreases.
- Based on the updated ability score, the system selects the next question with an appropriate difficulty level.
- This process continues for several questions until the quiz session is completed.

Weak Topic Detection

- While the quiz is running, the system also keeps track of the topics where the student makes mistakes.
- Whenever a student answers a question incorrectly, the system records the topic associated with that question.
- At the end of the quiz, the system analyzes all incorrect answers and counts how many mistakes were made in each topic.
- Topics with the highest number of incorrect answers are considered the student's **weak areas**.
- These weak topics are then used to generate personalized learning recommendations.

AI Integration

- To make the system more helpful, I integrated an AI model using the **Groq API**.
- Once the quiz is completed, the system sends the student's performance data to the AI model.
  This data includes the weak topics and the difficulty levels reached during the quiz.
- The AI model analyzes this information and generates a short **three-step study plan** that helps the student improve in the areas where they struggled the most.
- This allows the system to go beyond just showing quiz results and instead provide meaningful learning guidance.

API Design

- The system exposes several API endpoints that allow users to interact with the quiz engine.
- The **next question endpoint** retrieves a question based on the student's current ability score.
- The **submit answer endpoint** receives the student's answer and evaluates whether it is correct or incorrect.
- Finally, the **learning plan endpoint** analyzes the student's performance and returns a personalized study plan generated by the AI model.
- FastAPI automatically generates interactive API documentation using Swagger, which makes it easy to test these endpoints.

Example API Response

- When the system returns a question, it includes the question text and the available answer options.
- After the student submits an answer, the system returns a response indicating whether the answer was correct and the topic associated with that question.
- These responses allow the frontend or testing interface to display results to the user and update the quiz flow accordingly.

Demo

- In the demonstration, I will show how the system works in practice.
- First, I will show the MongoDB database containing the quiz questions.
- Next, I will start the FastAPI server and open the API documentation page.
- From there, I will demonstrate how the endpoints work by requesting questions and submitting answers.
- Finally, I will show how the system generates a personalized learning plan using the AI model based on the student's performance.

http://127.0.0.1:8000/docs#/default/learning_plan_learning_plan__session_id__get

Adaptive GRE Practice API 1.0 OAS 3.1

/openapi.json

An adaptive GRE practice testing system using FastAPI, MongoDB, and AI-powered learning insights.

Home

GET / Home

Session

POST /start-session Start Session

Quiz

GET /next-question/{session_id} Next Question

POST /submit-answer Submit Answer

AI Insights

GET /learning-plan/{session_id} Learning Plan

HomePage – Adaptive GRE practice

Schemas

AnswerRequest > Expand all object

HTTPValidationError > Expand all object

ValidationError > Expand all object

cloud.mongodb.com/v2/69af812f23bec2ac73ae0ba8#/explorer/69af81d45e5c24300557bb2c/adaptive_test/questions/find

ORGANIZATION Self > PROJECT Project 0

Data Explorer

- My Queries
- Data Modeling

CLUSTERS (1)

Search clusters

- Cluster0
  - adaptive_test
    - questions
    - sessions
  - admin
  - local
  - sample_mflix
  - test_database

questions | sessions

Cluster0 > adaptive_test > questions

View monitoring | Visualize Your Data

Documents 0 | Aggregations | Schema | Indexes 1 | Validation

Type a query: { field: 'value' } or Generate query

Explain | Reset | Find | Options

ADD DATA | UPDATE | DELETE | EXPORT CODE

25 | 1 – 20 of 20

questions

	_id ObjectId	question String	options Object	correct_answer String	difficulty Double	topic String
1	ObjectId('69b0054190867d5…	"If a number is increased…	{} 4 fields	"B"	0.2	"Algebra"
2	ObjectId('69b0054190867d5…	"What is the value of 18 …	{} 4 fields	"C"	0.2	"Arithmetic"
3	ObjectId('69b0054190867d5…	"A shop gives a 10% disco…	{} 4 fields	"A"	0.3	"Arithmetic"
4	ObjectId('69b0054190867d5…	"Solve for x: 7x = 63"	{} 4 fields	"C"	0.3	"Algebra"
5	ObjectId('69b0054190867d5…	"Vocabulary: Meaning of '…	{} 4 fields	"A"	0.3	"Vocabulary"
6	ObjectId('69b0054190867d5…	"What is the average of 1…	{} 4 fields	"B"	0.4	"Arithmetic"
7	ObjectId('69b0054190867d5…	"Solve: 2x + 5 = 17"	{} 4 fields	"B"	0.4	"Algebra"
8	ObjectId('69b0054190867d5…	"Vocabulary: Meaning of '…	{} 4 fields	"A"	0.4	"Vocabulary"
9	ObjectId('69b0054190867d5…	"If the perimeter of a sq…	{} 4 fields	"D"	0.5	"Arithmetic"
10	ObjectId('69b0054190867d5…	"Solve: x^2 = 64"	{} 4 fields	"C"	0.5	"Algebra"
11	ObjectId('69b0054190867d5…	"Vocabulary: Meaning of '…	{} 4 fields	"B"	0.5	"Vocabulary"
12	ObjectId('69b0054190867d5…	"A train travels 60 km in…	{} 4 fields	"D"	0.6	"Arithmetic"
13	ObjectId('69b0054190867d5…	"Solve: 3x + 4 = 19"	{} 4 fields	"C"	0.6	"Algebra"

Sample GRE Questions with options

System Status: All Good

©2026 MongoDB, Inc. Status Terms Privacy Atlas Blog Contact Sales

cloud.mongodb.com/v2/69af812f23bec2ac73ae0ba8#/explorer/69af81d45e5c24300557bb2c/adaptive_test/sessions/find

ORGANIZATION Self > PROJECT Project 0

Data Explorer

- My Queries
- Data Modeling

CLUSTERS (1)

Search clusters

- Cluster0
  - adaptive_test
    - questions
    - sessions
  - admin
  - local
  - sample_mflix
  - test_database

questions | sessions

Cluster0 > adaptive_test > sessions

View monitoring | Visualize Your Data

Documents 4 | Aggregations | Schema | Indexes 1 | Validation

Type a query: { field: 'value' } or Generate query

Explain | Reset | Find | Options

ADD DATA | UPDATE | DELETE | EXPORT CODE

25 | 1 – 9 of 9

```
_id: ObjectId('69afb495c20da87daabb67e1')
ability_score : 0.4
answers : Array (1)
current_question : ObjectId('69afb207131ac5e3814b3c60')

_id: ObjectId('69afb4dfad5eb9adb19094ab')
ability_score : 0.595253953859469
answers : Array (3)
current_question : ObjectId('69afb207131ac5e3814b3c60')

_id: ObjectId('69afb975bbdcba86708868f3')
ability_score : 1
answers : Array (7)
current_question : ObjectId('69afb207131ac5e3814b3c58')

1  _id: ObjectId('69afbbf37ede4a33ed17a7d9')                ObjectId
2  ability_score : 0.5238453929809397                       Double
3  answers : Array (20)                                     Array
4  current_question : 69afb207131ac5e3814b3c58              ObjectId
```

System Status: All Good

©2026 MongoDB, Inc. Status Terms Privacy Atlas Blog Contact Sales

http://127.0.0.1:8000/docs#/default/start_session_start_session_post

80%

POST `/start-session` Start Session

Start Session - UI _ FastAPI

Creates a new adaptive testing session

Parameters

Cancel

No parameters

Execute | Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/start-session' \
  -H 'accept: application/json' \
  -d ''
```

Request URL

```
http://127.0.0.1:8000/start-session
```

Server response

Code	Details
200	Response body

```
{
  "session_id": "69b02373d6956f5ebd4b188a",
  "ability_score": 0.5,
  "message": "Session started successfully"
}
```

Session ID

Download

Response headers

```
content-length: 102
content-type: application/json
date: Tue,10 Mar 2026 13:58:11 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	*No links*

Media type

application/json

Controls Accept header.

http://127.0.0.1:8000/docs#/default/next_question_next_question__session_id__get

80%

GET /next-question/{session_id} Next Question

Parameters

Cancel

Question/Session - UI _ FastAPI

Name	Description
session_id * required string (path)	69b02373d6956f5ebd4b188a

Execute | Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/next-question/69b02373d6956f5ebd4b188a' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/next-question/69b02373d6956f5ebd4b188a
```

Server response

Code	Details
200	Response body


```
{
  "question": "If the perimeter of a square is 36, what is the length of one side?",
  "options": {
    "A": "6",
    "B": "7",
    "C": "8",
    "D": "9"
  },
  "difficulty": 0.5
}
```


Download

Response headers

```
content-length: 143
content-type: application/json
date: Tue,10 Mar 2026 13:59:05 GMT
server: uvicorn
```

Responses

Code	Description	Links

http://127.0.0.1:8000/docs#/default/submit_answer_submit_answer_post

80%

POST /submit-answer Submit Answer

Answer Submission - UI _ FastAPI

Parameters

Cancel

No parameters

Request body required

application/json

Edit Value | Schema

```
{
  "session_id": "Enter_Your_Session_ID",
  "answer": "Option_Choosen"
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/submit-answer' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "session_id": "69b00720d6956f5ebd4b1889",
  "answer": "B"
}'
```

Request URL

```
http://127.0.0.1:8000/submit-answer
```

Server response

Code Details

http://127.0.0.1:8000/docs#/default/submit_answer_submit_answer_post

Edit Value | Schema

```
{
  "session_id": "69b02373d6956f5ebd4b188a",
  "answer": "D"
}
```

Session ID & Option Selector

Execute | Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/submit-answer' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "session_id": "69b02373d6956f5ebd4b188a",
  "answer": "D"
}'
```

Request URL

```
http://127.0.0.1:8000/submit-answer
```

Server response

Code	Details
200	Response body

```
{
  "correct": true,
  "new_ability": 0.6,
  "questions_answered": 1
}
```

Download

Answer Detail's

Response headers

```
content-length: 57
content-type: application/json
date: Tue,10 Mar 2026 14:00:35 GMT
server: uvicorn
```

http://127.0.0.1:8000/docs#/default/learning_plan_learning_plan__session_id__get

GET /learning-plan/{session_id} Learning Plan

Parameters

Cancel

Name	Description
session_id * required string (path)	69b02373d6956f5ebd4b188a

Execute | Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/learning-plan/69b02373d6956f5ebd4b188a' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/learning-plan/69b02373d6956f5ebd4b188a
```

Server response

Code	Details
200	Response body

LLM generated learning_plan @ GroqAPI's

```
{
  "weak_topics": {
    "Algebra": 2,
    "Vocabulary": 1
  },
  "learning_plan": [
    "Step 1: Dedicate 30 minutes each day to reviewing algebra concepts, focusing on weak areas (e.g., systems of equations, quadratic equations).",
    "Step 2: Allocate 60 minutes, 2 times a week, to practicing vocabulary with flashcards or online resources, aiming to learn 10-15 new words each session.",
    "Step 3: Solve 5-7 GRE-style algebra questions and 2-3 vocabulary questions daily to reinforce learned concepts and build endurance."
  ]
}
```

Download

Response headers

```
content-length: 495
content-type: application/json
date: Tue,10 Mar 2026 14:08:41 GMT
server: uvicorn
```

Responses

Code	Description	Links

AI Tools Used

- Used OpenAI ChatGPT as a coding assistant for generating FastAPI code, structuring modules, and debugging issues.
- Integrated Groq API (Grok model) to generate personalized learning plans based on weak topics.
- AI tools helped speed up development, but debugging, testing the adaptive logic, and system design decisions were handled manually.

Future Improvements

- Although this prototype works well, there are several ways it could be improved.
- One improvement would be implementing a more advanced **Item Response Theory (IRT)** model to estimate student ability more accurately.
- Another improvement would be storing long-term user performance data so that the system can track progress over time.
- Additional features such as user authentication, a frontend dashboard, and advanced analytics could also make the system more powerful and user-friendly.

Conclusion

- This project demonstrates how adaptive algorithms and AI can be combined to build smarter learning systems.
- By dynamically adjusting question difficulty and analyzing student performance, the system can estimate ability levels more accurately than traditional quizzes.
- The integration of an AI model further enhances the system by providing personalized study recommendations.
- Overall, this prototype shows how technology can be used to create more effective and personalized educational experiences.